Pacman Capture the Flag

Setup, Diagnosis and Three Competitive Strategies

Team `pacman-agent`

October 2026

Abstract

This document collects everything needed to compete in the Pacman Capture-the-Flag contest: a digest of the rules that matter for strategy, the root cause of the macOS simulation problem and how it was fixed, a cheat-sheet for running games locally, and three strategies of increasing strength and risk. `my_team.py` is still the organisers' original template. The strategies below describe what to build in it. A prototype of Strategy A won 40/40 test games against the baseline.

Contents

1 Rules That Shape Strategy **3**

2 Why the Simulation Did Not Work on the MacBook **3**
- 2.1 Symptom . . . 3
- 2.2 Diagnosis . . . 3
- 2.3 Secondary problems that made it look worse . . . 4
- 2.4 The fix (already applied) . . . 4

3 Running Simulations Locally **4**

4 Strategy Overview **5**

5 Strategy A — "Smart Reflex" **5**
- 5.1 Idea . . . 5
- 5.2 Map analysis (precomputed) . . . 5
- 5.3 Attacker . . . 6
- 5.4 Defender . . . 6
- 5.5 Role switching . . . 6
- 5.6 Prototype results . . . 6
- 5.7 Tuning knobs . . . 7

6 Strategy B — Tracking + Search **7**
- 6.1 Idea . . . 7
- 6.2 Belief tracking (exact inference over cells) . . . 7
- 6.3 Uses of the belief . . . 7
- 6.4 Pros and risks . . . 7

7 Strategy C — Adversarial Search / MCTS **8**
- 7.1 Idea . . . 8
- 7.2 Algorithm . . . 8
- 7.3 Pros and risks . . . 8

8 Roadmap and Submission Checklist **8**
8.1 Roadmap 8
8.2 Before every submission 9

1 Rules That Shape Strategy

The rules come from `CONTEST.md` and the constants in `capture.py`.

Rule	Consequence for our agents
Map split in two halves	You are a ghost (can eat) at home and a Pacman (can be eaten) on the enemy side.
Food scores only when brought home	Carrying food is risky. Bank it regularly instead of greedily eating everything.
Death drops carried food	Dying while carrying 10 dots throws away 10 points of work and gives the defender time.
Win condition	Return 28 dots ($\frac{60}{2} - 2$), or lead after 1200 moves (300 per agent).
Ranking: win 3, tie 1, loss 0	A 0–0 draw is nearly worthless. Pure defence is not enough.
Power capsule	Enemy ghosts are scared for 40 moves and can be eaten (they respawn at start).
Partial observability	Exact enemy positions only within Manhattan distance 5. Otherwise noisy distance ± 6.
Time limits	15 s setup, 1 s per move (3rd warning forfeits), any move > 3 s forfeits.
Hardware	1.7 GHz, 1.7 GB RAM, so budget ≈ 0.3 s per move to be safe.
Layouts	Default plus random symmetric mazes. Either side may start, so never hard-code a map.
No threads, no printing	Threading is disqualifying. stdout/stderr is published with the results.

Only one file is submitted

The tournament runs `python3 capture.py -r teams/<a>/my_team.py -b teams/<b>/my_team.py`. `create_team()` is called *without* extra options, so its default arguments are the team that plays. Helper modules must be imported after appending the file's own folder to `sys.path`.

2 Why the Simulation Did Not Work on the MacBook

2.1 Symptom

Launching a game with graphics (`python capture.py ...`) never showed a usable window: the process appeared frozen. Headless runs *seemed* to hang too, because each game took around 40 seconds with almost no output.

2.2 Diagnosis

1. The original `venv` was created from `/usr/bin/python3`. On macOS that is the **Python 3.9.6 bundled with Apple's Command Line Tools** (`venv/pyvenv.cfg`: `home = /Library/Developer/CommandLineTools/usr/bin`).
2. That Python links against Apple's **system Tcl/Tk 8.5** framework (`tkinter.TkVersion == 8.5`). Apple deprecated it years ago and no longer maintains it.

3. The game's graphics layer (`graphics_utils.py`) starts with `tkinter.Tk()`. A one-line probe, `tkinter.Tk().update()`, under that interpreter on **macOS 26.6** *never returned*. It spun at 100% CPU and grew to about 1 GB of RAM. The old Tk 8.5 cannot initialise its Cocoa window on current macOS.
4. Homebrew's **Python 3.14 with Tk 9.1** (`python-tk@3.14`) was already installed. The same probe returned instantly, and a full graphical game ran correctly.

Root cause in one sentence

The virtual environment was built on Apple's outdated Command-Line-Tools Python, whose Tk 8.5 GUI toolkit hangs on macOS 26. The game code is fine; the interpreter's GUI library is broken.

2.3 Secondary problems that made it look worse

Problem	Explanation and fix
40 s per "headless" game	`--delay-step` defaults to 0.03 s, and $1200 \times 0.03 = 36$ s of pure sleep. Pass `--delay-step 0`. A game then takes under 1 s.
"No teams found!"	`-r baseline_team` is resolved relative to the *current folder*, so it only works inside `src/contest`. Use full paths.
`scripts/play.sh` broken	It references `myTeam.py` (doesn't exist) and the broken `runner.py`. Use the README command instead.
`scripts/config.sh`	Re-adds the submodule from a *different* fork. Do not run it.
`--delay` in CONTEST.md	The real flag is `--delay-step`.
`venv/lib 2` folder	A Finder/iCloud duplicate. The project lives on an iCloud-synced Desktop, which can corrupt virtual environments. Prefer e.g. `~/code/`.

2.4 The fix (already applied)

```
cd pacman-agent
rm -rf venv
/opt/homebrew/bin/python3.14 -m venv venv # Homebrew Python with Tk 9.1
source venv/bin/activate
pip install --upgrade pip
pip install -e pacman-contest # the 'contest' package
pip install frozendict numpy
python -c "import tkinter; print(tkinter.TkVersion)" # prints 9.1
```

If Homebrew's Tk is ever missing: `brew install python@3.14 python-tk@3.14`. Note that Homebrew `python@3.13` on this machine has *no* Tk module.

3 Running Simulations Locally

As in the README: activate the venv, then run everything from the engine folder (`source venv/bin/activate`, then `cd pacman-contest/src/contest`).

Goal	Command
Watch a game	`python capture.py -r baseline_team -b ../../../my_team.py`
Random map	add `-l RANDOM13`
Other map	add `-l officeCapture` (any name in `layouts/`)
Slow motion	add `--delay-step 0.1`
Fast, no graphics	add `-q --delay-step 0`
Ten games in a row	add `-n 10 -q --delay-step 0`
Your team as blue	`python capture.py -r ../../../my_team.py -b baseline_team`
Record a game	add `--record`, then `--replay www/contest_default/replays/match_0.replay`
Play yourself	add `--keys0` (arrow keys / WASD control red agent 0)
All options	`python capture.py --help`

Testing discipline

- Always test as **both red and blue** and on **random layouts**. Use `-l RANDOM` and swap `-r`/`-b`.
- Before replacing `my_team.py`, keep a copy of the current one and play the new version against it.
- Keep a few strong opponent versions around. Beating only the baseline proves little.

4 Strategy Overview

	A: Smart Reflex	B: Tracking + Search	C: Adversarial Search
Core idea	Rule-based goals + Dijkstra paths	A + belief tracking + A* intercepts	B + alpha-beta / MCTS in close fights
Strength	Solid	Strong	Highest ceiling
Effort	Low (done)	Medium	High
Timeout risk	None (7 ms)	Low	Needs time guards
Main weakness	Blind to unseen enemies	Hand-tuned thresholds	Hard to tune, slow

Recommended path: submit A now, then layer B on top, and add C only for close-combat situations. Benchmark every step against the previous version.

5 Strategy A — "Smart Reflex"

5.1 Idea

One **attacker** and one **defender**. Each turn, each agent picks a *goal* with simple rules, then plans the cheapest path to it with Dijkstra. Cells near dangerous ghosts get a heavy cost, and cells adjacent to a ghost are blocked. Everything that can be precomputed is computed once in `register_initial_state`.

5.2 Map analysis (precomputed)

- **Home border:** the column of our cells next to the middle line. Going home means reaching *any* border cell, not the spawn point.
- **Dead ends:** cells with one open neighbour are pruned repeatedly. Each pruned cell gets a *depth* (moves to escape the pocket) and an *entrance* (the exit cell).

5.3 Attacker

Algorithm 1 Attacker goal selection (each turn)

if food left ≤ 2 **or** (carrying > 0 **and** moves left $\leq$ distance home $+2$) **then**
    return path to home border
end if
if a dangerous ghost is within 5 **then**
    if some capsule is closer to us than to every ghost **then**
        return path to that capsule
    end if
    if carrying > 0 **or** ghost within 2 **then**
        return path to home border
    end if
end if
if carrying $\geq$ limit (6, or 4 when home is far) **then**
    return path to home border
end if
$S \leftarrow$ food where (no ghost is closer than us) **and** (if in a pocket: we can reach it and escape before a ghost reaches the entrance)
Remove food near the teammate's target (if both attack)
return Dijkstra path to nearest food in S, with ghost-proximity costs

A ghost counts as *dangerous* unless its scared timer exceeds its distance to us. While capsule power lasts, the attacker ignores those ghosts and eats greedily.

5.4 Defender

- **Visible invader:** chase it along home cells only, so the defender never becomes a Pacman.
- **Scared:** shadow the invader at distance 2 instead of touching it, then strike when the timer ends.
- **Invisible invader:** food that vanished since the last observation reveals where it is. Go there.
- **No invader:** patrol the border cell with the smallest average distance to the food and capsules we still defend.

5.5 Role switching

- The attacker, while still a ghost at home, eats any invader within 4 steps ("free kill").
- Late game (fewer than 60 moves each) and ahead: the attacker drops back to defend the lead.
- Second half, not winning, and no invaders: the defender joins the attack.

5.6 Prototype results

Opponent	Games	W / T / L	Avg. margin	Max move time
`baseline_team`*	40	40 / 0 / 0	+14	7 ms

*10 fixed + 10 random layouts, each played as red and as blue.

5.7 Tuning knobs

`CARRY_LIMIT` (6), `DANGER_RADIUS` (5), `SAFETY_MARGIN` (2),
the ghost cost weights in `ghost_cost_fn`, and the role-switch thresholds (60 / 200 moves). Change one at a time and re-test against the previous version.

6 Strategy B — Tracking + Search

6.1 Idea

Strategy A is blind beyond distance 5. Strategy B maintains a **probability distribution over each enemy's position**, so both agents act on where enemies *probably* are.

6.2 Belief tracking (exact inference over cells)

Algorithm 2 Per-enemy belief update (each turn)

Init: all mass on the enemy's start cell
Elapse time: spread each cell's mass uniformly over itself and its neighbours
Observe: multiply by P(noisy dist | true dist) via `get_distance_prob`
Zero out cells inconsistent with `is_pacman` (which side the enemy is on)
Zero out cells within 5 of us or our teammate (we would see it there)
if enemy visible **then**
    put all mass on its position
end if
if our food vanished **then**
    put mass on that food cell
end if
if enemy just died (position → start) **then**
    reset to start
end if
Normalise (if all zero, reset to uniform over its side)

Cost is O(cells) per enemy per turn, a few milliseconds. Both agents share one belief through a module-level dictionary.

6.3 Uses of the belief

- **Defender intercepts:** instead of chasing from behind, move to the border chokepoint on the path between the invader's most likely cell and our densest food.
- **Attacker safety:** treat high-probability cells as soft ghost positions in the Dijkstra cost, so it avoids ghosts it cannot see yet.
- **Entry point choice:** cross the border where expected ghost distance is largest.
- **Food clustering:** plan "eat k nearby dots, then escape" routes with A*, scoring $\frac{\text{dots}}{\text{route length}}$ minus risk.
- **Two-agent split:** assign food clusters (e.g. top/bottom half) so attackers never compete.

6.4 Pros and risks

Large gain against mid-level teams that slip in unseen. The risk is mostly bugs (beliefs collapsing to zero), so keep the reset rule and visualise beliefs with `self.debug_draw`.

7 Strategy C — Adversarial Search / MCTS

7.1 Idea

Reflex rules handle most of the game, but life-or-death moments (escaping a ghost, cornering a Pacman) are where games are decided. When an enemy is within about 6 steps, switch to **look-ahead search** over the real game model (`game_state.generate_successor`).

7.2 Algorithm

- **Players:** us plus the nearest one or two enemies. The teammate follows its reflex policy.
- **Unseen enemies:** place them at their most likely belief cell (from B).
- **Search:** alpha-beta, iterative deepening (depth 2, 4, 6, ...), or MCTS with UCT for roughly 0.4 s.
- **Leaf evaluation:** score, carried food, distance home, $-\infty$ if eaten while carrying, + for eating an invader, food distance — reuse A/B features.
- **Time guard:** check `time.time()` in the search loop and return the best move found so far. Fall back to the reflex move if no search completes.

Algorithm 3 Hybrid controller

```
t0 ← now
if nearest relevant enemy (seen or believed) within 6 then
    a ← iterative-deepening alpha-beta until now − t0 > 0.4 s
else
    a ← Strategy B action
end if
return a
```

7.3 Pros and risks

Highest skill ceiling. The searcher finds escapes and traps that hand-written rules miss. Risks: the grading machine is slower than a MacBook (budget conservatively), `generate_successor` is expensive (cache successors), and the evaluation needs careful tuning. Test with the `-i` option and watch the output for `took too long` warnings.

8 Roadmap and Submission Checklist

8.1 Roadmap

1. **First:** build Strategy A in `my_team.py`. It is safe and beats the baseline.
2. Keep a copy of it. Every new version must beat it as red and as blue.
3. Add belief tracking (B), visualise it, and use it in the defender first, where the gain is biggest.
4. Add belief-aware attacker costs and food-cluster routes.
5. Add time-boxed search (C) for close encounters only.
6. Study tournament replays of losses. Each loss usually points at one missing rule.

8.2 Before every submission

- ☐ No `print` statements or debug drawing left in `my_team.py`.
- ☐ `create_team` defaults select the intended agents.
- ☐ Many games with `-n 10 -l RANDOM`: zero errors, zero time warnings.
- ☐ Works as both red and blue, on random maps.
- ☐ `TEAM.md` filled in.
- ☐ Any helper modules imported via the `sys.path` trick.

